RAPPORT DE SÉCURITÉ APPLICATIVE

Phase 1 — SAST : Analyse Statique du Code Source

Projet	wusu-backend
Date d'analyse	29 April 2026
Analyste	Équipe Sécurité Applicative
Statut	**⚠SECRET EXPOSÉ — ACTION REQUISE**

⍰ Résumé Exécutif

Cette analyse SAST de la Phase 1 a porté sur 211 fichiers du projet wusu-backend. Quatre outils spécialisés ont été déployés : Semgrep (vulnérabilités code), TruffleHog (secrets exposés), ESLint/SonarJS (qualité) et Snyk (dépendances). Le bilan est globalement satisfaisant — aucune faille applicative dans le code source, aucune dépendance vulnérable — mais une fuite de secret critique (clé API Twilio) a été identifiée et doit être traitée en urgence absolue avant tout déploiement.

⍰ Tableau de Bord — Synthèse des Résultats

Outil	Périmètre	Résultat	Niveau de risque
Semgrep OSS	211 fichiers / 453 règles	0 finding	**VERT — AUCUN RISQUE**
TruffleHog	Git history + filesystem	1 secret vérifié (Twilio)	**ROUGE — CRITIQUE**
ESLint + SonarJS	Codebase JS/TS	0 erreur ESLint	**VERT — AUCUN RISQUE**
Snyk SCA	package-lock.json	0 vulnérabilité	**VERT — AUCUN RISQUE**

1. Analyse Statique du Code — Semgrep OSS

⍰ Outil utilisé : Semgrep OSS

Semgrep est un outil d'analyse statique de code source (SAST) open-source. Il analyse syntaxiquement les fichiers de code à la recherche de patterns correspondant à des règles de sécurité prédéfinies (injections, XSS, secrets hardcodés, mauvaises pratiques cryptographiques, etc.) sans exécuter l'application.

Commande exécutée :

```
semgrep scan --config auto
```

⍰ Résultat de la commande

```
┌──── ○○○ ────┐
│ Semgrep CLI │
└─────────────┘

Scanning 211 files (only git-tracked) with:

✓ Semgrep OSS
  ✓ Basic security coverage for first-party code vulnerabilities.

✗ Semgrep Code (SAST)        [Non activé — compte pro requis]
✗ Semgrep Supply Chain (SCA) [Non activé — compte pro requis]

┌──────────────┐
│ Scan Summary │
└──────────────┘
✅ Scan completed successfully.
 • Findings:        0  (0 blocking)
 • Rules run:       453
 • Targets scanned: 211
 • Parsed lines:    ~100.0%
 • Scan skipped:
   ◦ Files matching .semgrepignore patterns: 12
 • Scan was limited to files tracked by git

Ran 453 rules on 211 files: 0 findings.

□ Missed out on 1738 pro rules since you aren't logged in!
```

⍰ Interprétation experte

Points positifs

- 453 règles de sécurité OSS appliquées : couverture des injections SQL/NoSQL, XSS, SSRF, RCE, mauvaises pratiques cryptographiques, etc.
- 211 fichiers analysés avec un taux de parsing de 100 % — aucun fichier syntaxiquement invalide.
- 0 finding : aucune vulnérabilité détectée dans le périmètre OSS.

Limites identifiées — actions recommandées

- Couverture partielle : Semgrep OSS n'a appliqué que 453/2 191 règles disponibles. Les 1 738 règles Pro (SAST avancé + Supply Chain) nécessitent un compte gratuit.

- Action — Créer un compte Semgrep gratuit (`semgrep login`) pour activer les règles Pro et relancer l'analyse.
- 12 fichiers exclus via .semgrepignore — vérifier qu'il ne s'agit pas de fichiers sensibles exclus par erreur.

✅VERDICT : AUCUNE VULNÉRABILITÉ DÉTECTÉE (périmètre OSS)

2. Détection de Secrets Exposés — TruffleHog

⍰ Outil utilisé : TruffleHog v3.95.2

TruffleHog est un outil de détection de secrets dans le code source et l'historique Git. Contrairement à un simple grep, il utilise des détecteurs spécialisés par service (AWS, Twilio, Stripe, GitHub, etc.) et vérifie la validité des secrets trouvés en temps réel auprès des APIs correspondantes.

Commandes exécutées :

```
# Scan de l'historique Git complet
trufflehog git file://. --only-verified

# Scan du système de fichiers
trufflehog filesystem .
```

⍰ Résultats — Secret Twilio détecté et vérifié

```
□□□  TruffleHog. Unearth your secrets. □□□

✅Found verified result □□
Detector Type:     Twilio
Decoder Type:      PLAIN
Raw result:        AC804f552440c18a59401d4edce3b26c98
Rotation_guide:    https://howtorotate.com/docs/tutorials/twilio/

Commit:            9c53a1ca6c22089928d4af1fd505056a46ce188d
Email:             bannalice-ui <b.annalice@wusu.io>
File:              src/main/resources/application-dev.yml
Line:              25
Timestamp:         2026-04-28 22:19:05 +0000

finished scanning — verified_secrets: 1 | unverified_secrets: 0
scan_duration: 1.42s
```

⍰ Analyse de la vulnérabilité

Attribut	Valeur
Type de secret	Clé d'authentification Twilio (Account SID)
Statut	**⚠VÉRIFIÉ — La clé est active et fonctionnelle**
Fichier impacté	src/main/resources/application-dev.yml ligne 25
Commit source	9c53a1ca6c22089928d4af1fd505056a46ce188d
Auteur du commit	b.annalice@wusu.io

⚠Impact et risques

Une clé Twilio Account SID active exposée dans le code source représente une menace de niveau CRITIQUE pour les raisons suivantes :

- Facturation frauduleuse : un attaquant peut envoyer des millions de SMS/appels au débit du compte Twilio de l'organisation, générant des coûts pouvant atteindre des dizaines de milliers d'euros.

- Usurpation d'identité : envoi de SMS frauduleux (phishing, smishing) au nom de l'organisation depuis ses propres numéros.
- Exfiltration de données : accès aux logs d'appels et SMS, pouvant révéler des numéros de téléphone d'utilisateurs et des données personnelles (RGPD).
- Persistance dans l'historique Git : même si le fichier est modifié, le secret reste présent dans l'historique Git et accessible via `git log`.

⚠ Plan de remédiation OBLIGATOIRE

Actions à effectuer dans les 24 heures :

Étape 1 — Révoquer immédiatement la clé Twilio

- Se connecter à https://console.twilio.com
- Régénérer l'Auth Token et désactiver la clé exposée
- Vérifier les logs d'utilisation pour détecter toute activité anormale

Étape 2 — Purger le secret du dépôt Git

```
# Option 1 — BFG Repo Cleaner (recommandé)
bfg --replace-text secrets.txt
git reflog expire --expire=now --all
git gc --prune=now --aggressive

# Option 2 — git-filter-repo
git filter-repo --path src/main/resources/application-dev.yml --invert-paths
```

Étape 3 — Migrer vers les variables d'environnement

- Ne jamais stocker de secrets dans les fichiers de configuration versionnés
- Utiliser des variables d'environnement (`.env`) avec un fichier `.gitignore` adéquat
- Envisager un gestionnaire de secrets (HashiCorp Vault, AWS Secrets Manager, Doppler)

Étape 4 — Ajouter une protection préventive

- Configurer un pre-commit hook TruffleHog pour bloquer les commits avec des secrets
- Activer GitHub/GitLab Secret Scanning sur le dépôt distant

⚠ VERDICT : CRITIQUE — SECRET TWILIO ACTIF EXPOSÉ — RÉVOCATION IMMÉDIATE REQUISE

3. Qualité du Code & Dette Technique — ESLint + SonarJS

⍰ Outils utilisés : ESLint + eslint-plugin-sonarjs

ESLint est le linter de référence pour JavaScript/TypeScript. Combiné au plugin eslint-plugin-sonarjs (développé par SonarSource), il détecte non seulement les erreurs de syntaxe mais aussi les problèmes de maintenabilité, de logique, et de dette technique : code dupliqué, fonctions trop complexes, conditions cognitives trop imbriquées, variables inutilisées, etc.

Commandes exécutées :

```
npm install eslint-plugin-sonarjs --save-dev
npx eslint . --format stylish
npx eslint . --format html > rapport-qualite.html
```

⍰ Résultat de l'installation

```
mac@Leroy-MacBook-Air wusu-backend % npm install eslint-plugin-sonarjs --save-dev

added 12 packages, and audited 84 packages in 4s

19 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

mac@Leroy-MacBook-Air wusu-backend % npx eslint . --format stylish
mac@Leroy-MacBook-Air wusu-backend % npx eslint . --format html > rapport-qualite.html
mac@Leroy-MacBook-Air wusu-backend %
```

⍰ Interprétation experte

La commande `npx eslint . --format stylish` n'a produit aucune sortie visible dans le terminal. En ESLint, l'absence de sortie signifie qu'aucun warning ni aucune erreur n'a été détecté selon les règles configurées dans le projet.

Analyse des résultats

- 0 erreur ESLint : aucune violation des règles de style, de syntaxe ou de logique configurées.
- 0 vulnérabilité dans les packages installés : l'installation de eslint-plugin-sonarjs confirme un écosystème npm sain.
- Rapport HTML généré : rapport-qualite.html produit pour archivage et revue visuelle détaillée.

Points d'attention

- L'absence d'erreurs dépend de la configuration .eslintrc (ou eslint.config.js). Si les règles SonarJS ne sont pas activées dans ce fichier, le plugin installé ne sera pas effectif.
- Il est recommandé de vérifier que les règles sonarjs/* sont bien référencées dans la configuration ESLint et d'activer au minimum : sonarjs/cognitive-complexity, sonarjs/no-duplicate-string, sonarjs/no-identical-functions.

✅VERDICT : QUALITÉ CODE CONFORME — 0 ERREUR ESLINT DÉTECTÉE

4. Analyse des Dépendances Open Source — Snyk SCA

⍰ Outil utilisé : Snyk CLI (SCA — Software Composition Analysis)

Snyk est une plateforme de sécurité DevSecOps spécialisée dans l'analyse des dépendances open source (SCA). Elle croise la liste des packages utilisés avec des bases de données de CVE (Common Vulnerabilities and Exposures) pour identifier les versions vulnérables, les chemins d'exploitation ("reachable paths") et proposer des versions de mise à jour.

Commandes exécutées :

```
npm install -g snyk
snyk auth                    # Authentification OAuth via navigateur
snyk test                   # Analyse des dépendances
```

⍰ Résultat de l'analyse

```
Testing /Users/mac/Desktop/wusu-backend...

Organization:      leroy8889
Package manager:   npm
Target file:       package-lock.json
Project name:      package.json
Open source:       no
Project path:      /Users/mac/Desktop/wusu-backend
Licenses:          enabled

✓ Tested /Users/mac/Desktop/wusu-backend for known issues,
  no vulnerable paths found.

Tip: Detected multiple supported manifests (1),
     use --all-projects to scan all of them at once.

Next steps:
- Run `snyk monitor` to be notified about new related vulnerabilities.
- Run `snyk test` as part of your CI/test.
```

⍰ Interprétation experte

Points positifs

- 0 chemin vulnérable détecté : aucune dépendance directe ou transitive ne présente de CVE connue exploitable.
- Analyse des licences activée : vérification de la conformité des licences open source (GPL, MIT, Apache, etc.) — aucune alerte.
- Authentification OAuth réussie : connexion au moteur de règles Snyk garantissant l'utilisation de la base de données CVE la plus récente.

Limites identifiées — actions recommandées

- Manifeste unique analysé : Snyk a détecté plusieurs manifestes potentiels (package.json + pom.xml si projet hybride Java/Node). Relancer avec `snyk test --all-projects` pour couvrir toutes les dépendances.

- Surveillance continue : configurer `snyk monitor` en CI/CD pour être alerté dès qu'une CVE affecte une dépendance actuellement saine.
- Backend Java : si le projet wusu-backend contient également un pom.xml Maven, les dépendances Java ne sont couvertes que si `--all-projects` est activé.

✅VERDICT : AUCUNE DÉPENDANCE VULNÉRABLE DÉTECTÉE

5. Recommandations Globales & Plan d'Action

⍰ Matrice de priorité des actions

Priorité	Action	Délai	Responsable
P0 ⍰	Révoquer la clé Twilio exposée	< 24h	Lead Dev + DevSecOps
P0 ⍰	Purger le secret de l'historique Git	< 24h	Lead Dev
P1 ⍰	Activer Semgrep Pro (compte gratuit)	< 1 semaine	DevSecOps
P1 ⍰	Ajouter pre-commit TruffleHog	< 1 semaine	DevSecOps

⍰ Recommandations pour la Phase 2

La Phase 1 SAST ayant été complétée, voici les analyses complémentaires recommandées pour couvrir l'ensemble du cycle de sécurité applicative :

- Phase 2 — DAST (Dynamic Application Security Testing) : analyse de l'application en cours d'exécution avec OWASP ZAP ou Burp Suite pour détecter les failles en conditions réelles (injection, authentification, gestion des sessions).
- Phase 3 — Tests de pénétration manuels : validation des vulnérabilités détectées et recherche de failles logiques complexes non détectables automatiquement.
- Phase 4 — Intégration CI/CD : automatiser l'ensemble des outils SAST (Semgrep, TruffleHog, Snyk) dans la pipeline GitHub/GitLab Actions.

6. Conclusion

L'analyse SAST Phase 1 du projet wusu-backend présente un bilan globalement positif. Le code source ne contient pas de vulnérabilités applicatives connues dans le périmètre analysé, les dépendances open source sont saines, et la qualité du code respecte les standards ESLint définis.

Cependant, la découverte d'une clé d'authentification Twilio active et vérifiée dans l'historique Git constitue un incident de sécurité de niveau CRITIQUE qui doit être traité en priorité absolue, indépendamment de tout autre développement ou déploiement.

Ce rapport servira de référence pour l'équipe de développement afin de corriger les points identifiés. Une nouvelle analyse SAST devra être conduite après application des corrections pour valider la remédiation.

Rédigé par	Validé par
Équipe Sécurité Applicative	RSSI / Responsable Technique